

Cuarta Papaya

Memoria de desarrollo de extensión para Godot

Visualizador de modelos en 4D

Andrés García Navarro

Carmen Gómez Becerra

Índice

Índice.....	2
Resumen.....	3
Diagrama de clases.....	3
Proyección 4D a 3D.....	4
Rotación en 4D.....	5
Física en 4D.....	5
Apéndice.....	6
Escenas principales.....	6
Escena de físicas.....	6
Escena de visualizador en cuatro dimensiones de figuras tridimensionales.....	6
Explicación breve de cada clase (GDExtension).....	6
Node4D.....	6
MeshInstance4D.....	6
Mesh4D.....	6
Tesseract.....	7
GenericMesh4D.....	7
OffFileReader.....	7
Clases relacionadas con las físicas.....	7
Explicación breve de cada clase (GDScript).....	7
Clases relacionadas con la escena de físicas.....	7
Clases relacionadas con la escena del visualizador de figuras.....	7
Otras clases.....	8
Bibliografía.....	8
Construcción de mallas:.....	8
Construcción de hipercubo:.....	9
Rotaciones:.....	9
Visualizador de mallas 4D (Escena proyecto de Godot):.....	9
Diagrama de clases.....	9

Resumen

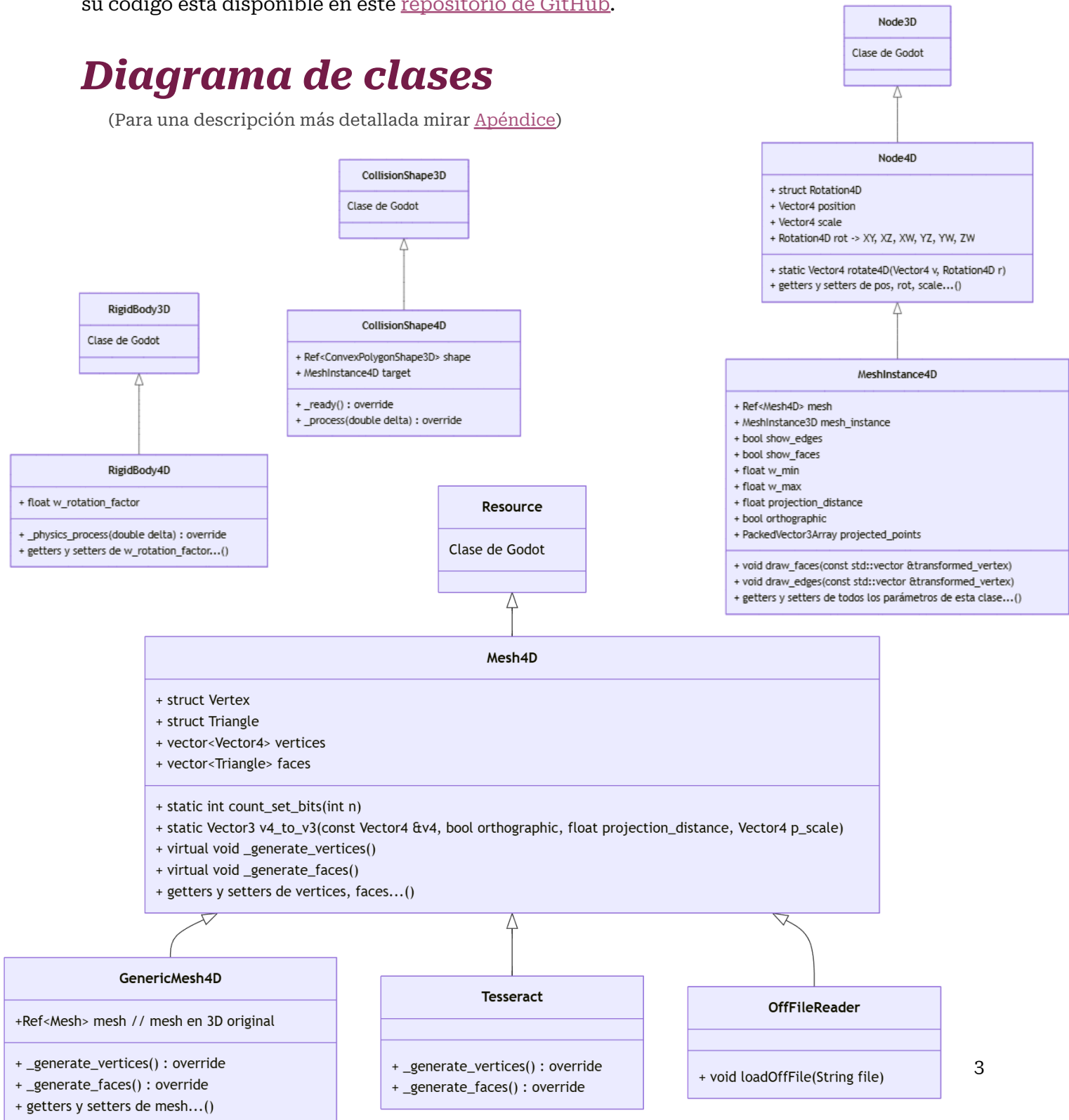
Este proyecto consiste en el desarrollo de una extensión para el motor de desarrollo Godot que proporciona herramientas para crear nodos en 4 dimensiones así como su renderizado. Estamos aplicando esta extensión en un proyecto donde se mostrará el efecto del 4D a través de un entorno en cuatro dimensiones del cual se podrá ver una única sección de esta cuarta dimensión.

Para desarrollar esta extensión estamos usando [Godot 4.4.1](#) (Versión de los laboratorios) y su correspondiente plantilla oficial de [GDExtension](#).

Se puede ver la extensión desarrollada y el proyecto que la incluye en el enlace a [itch.io](#), y su código está disponible en este [repositorio de GitHub](#).

Diagrama de clases

(Para una descripción más detallada mirar [Apéndice](#))



Proyección 4D a 3D

Para poder proyectar un vértice 4D a 3D hemos implementado dos métodos de proyección,

1. **Modo ortográfico**, donde se desecha el componente W y solo se aplica la escala del modelo.
2. **Modo de perspectiva**: El más relevante. Colapsamos la dimensión W mediante una división en perspectiva dependiente de la profundidad, escalando usando el factor: $(d/(d-w))$. Esta implementación está basada en la cámara de [S.R.Hollasch](#) pero mientras que Hollasch tiene una cámara 4D libre, la nuestra está fija y alineada con el eje W, esto quiere decir que la cámara está fija en el espacio 4D, mientras que la cámara 3D si es libre, en nuestro caso son los propios objetos los que se mueven.

```
static Vector3 v4_to_v3(const Vector4 &v4, bool orthographic,
    float projection_distance, Vector4 p_scale = Vector4(1, 1, 1, 1))
{
    float scale = 1.0f;
    if (!orthographic)
    {
        float denom = projection_distance - v4.w;
        if (Math::abs(denom) < 1e-6f) denom = 1e-6f;
        scale = projection_distance / denom;
    }

    return Vector3(v4.x * p_scale.x * scale,
        v4.y * p_scale.y * scale,
        v4.z * p_scale.z * scale);
}
```

Para dibujar las caras, aplicamos un recorte para cada pòligono delimitado por los planos de W que delimitan el rango visible $[w_{min}, w_{max}]$. Utilizamos el algoritmo de [Sutherland-Hodgman](#), consiguiendo así una “rebanada” de malla visible

```
static std::vector<Vector4> clip_polygon_w(const std::vector<Vector4> &polygon,
    float w_plane, bool keep_above)
{
    std::vector<Vector4> result;
    if (polygon.empty()) return result;

    for (size_t i = 0; i < polygon.size(); i++) {
        Vector4 current = polygon[i];
        Vector4 next = polygon[(i + 1) % polygon.size()];

        bool curr_in = keep_above ?
            (current.w >= w_plane) : (current.w <= w_plane);
        bool next_in = keep_above ?
            (next.w >= w_plane) : (next.w <= w_plane);

        if (curr_in) {
            result.push_back(current);
            if (!next_in) result.push_back(interp_w(current, next, w_plane));
        }
        else if (next_in) result.push_back(interp_w(current, next, w_plane));
    }
    return result;
}
```

Rotación en 4D

Una rotación viene definida por seis bloques de “rotaciones simples” que se definen como una rotación dentro de un único plano que deja fijo el plano ortogonal a él.

El bloque **XY** gira el plano **xy** y deja quieto **zw**,

El bloque **XZ** gira el plano **xz** y deja quieto **yw**,

...

```
static inline Vector4 rotate4D(Vector4 v, Rotation4D r) {
    float x = v.x, y = v.y, z = v.z, w = v.w;
    float nx, ny, nz, nw;
    // XY
    nx = x * cos(r.xy) - y * sin(r.xy);
    ny = x * sin(r.xy) + y * cos(r.xy);
    x = nx; y = ny;
    // XZ
    nx = x * cos(r.xz) - z * sin(r.xz);
    nz = x * sin(r.xz) + z * cos(r.xz);
    x = nx; z = nz;
    // XW
    nx = x * cos(r.xw) - w * sin(r.xw);
    nw = x * sin(r.xw) + w * cos(r.xw);
    x = nx; w = nw;

    (...)

    return Vector4(x, y, z, w);
}
```

Esto es en esencia un equivalente 4D de los [ángulos de Euler](#), una composición secuencial de giros elementales.

Física en 4D

De nuevo, para simplificar, nuestra física no es una física real. Podemos diferenciar esta simulación física en dos partes:

- **CollisionShape4D**: Escanea la jerarquía en busca de un *MeshInstance4D* hermano del que poder coger los vértices proyectados y así ajustar la malla de colisión a estos vértices.
- **RigidBody4D**: Se toma prestada la velocidad angular calculada por el motor físico 3D de Godot y se integra en los planos de rotación que involucran a la componente W (xw, yw, zw), Esto es un truco visual para introducir rotación en la cuarta dimensión a partir de la física tridimensional, sin necesidad de un motor físico 4D real.

```
node4d->set_rot_xw(rot.xw + ang_vel.x * w_rotation_factor * delta);
node4d->set_rot_yw(rot.yw + ang_vel.y * w_rotation_factor * delta);
node4d->set_rot_zw(rot.zw + ang_vel.z * w_rotation_factor * delta);
```

Apéndice

Escenas principales

Escena de físicas

En ella se puede interactuar con politopos cuatridimensionales y ver cómo funcionan las físicas entre ellos por medio de arrastrar y soltar con el clic derecho del ratón.

Escena de visualizador en cuatro dimensiones

En ella se pueden visualizar, interactuar y ajustar parámetros de diferentes polícoros.

Escena de visualizador en cuatro dimensiones de figuras tridimensionales

En ella se pueden visualizar, interactuar y ajustar parámetros de politopos tridimensionales representados en la cuarta dimensión.

Explicación breve de cada clase (GDExtension)

Node4D

El nodo 4d es una clase hija de la clase de Godot “*Node3D*”, es la encargada de manejar la posición y rotación de los objetos, además cuenta con el método estático “*rotate4D()*” que es el encargado de aplicar una rotación a un punto del mundo.

Idealmente todas las clases 4D que hemos creado deberían heredar de este nodo para mantener la arquitectura interna de Godot. Por practicidad, hemos decidido no hacerlo y utilizar las clases 3D existentes de Godot, ya que, al proyectar a 3D, son las clases que vienen preparadas.

MeshInstance4D

Es la clase encargada de renderizar los vértices y caras de una mesh en el mundo 4D proyectadas en 3D, cuenta con parámetros para personalizar la visualización:

- Mostrar las caras proyectadas y recortadas en el espacio visible del eje W.
- Mostrar el Wireframe o aristas del modelo 4D sin cortes por rango de W visible.
- Proyección en vista “ortográfica” (Sin escalado por la coordenada W).
- Distancia de proyección en el eje W.
- Rango visible del eje W.

Mesh4D

Esta es una clase base-recurso que contiene información de los vértices y caras de los modelos utilizados. Además de contener información, se usa como constructora de mallas, cuando se instancia un *Mesh4D* se llama a los métodos que generan la malla en su constructora.

También cuenta con helpers estáticos para proyectar un vértice en **4D a 3D**, un contador de bits activos para generar meshes utilizando **operaciones en bits** (Mirar implementación de

Tesseract), además de un **struct Vertex** (contiene un vértice y su índice) y un **struct Triangle** (Contiene 3 índices que construyen un triángulo)

Mesh4D es padre de 3 clases que constituyen los vértices y caras de una malla de forma diferente.

Tesseract

Un **teseracto**, **hipercubo** u **octacoro** es un tipo de polítopo de 4 dimensiones², dicho de otro modo, es el análogo de un cubo pero en 4D.

Se implementa añadiendo los vértices y caras de forma *hardcodeada* ya que está pensada como una clase primitiva sencilla.

GenericMesh4D

Esta clase transporta mallas en 3D al mundo en 4D asignándoles un valor base a su componente W de 0. A éstas mallas en 3D (Ahora 4D) se les puede aplicar posiciones, rotaciones y escalas en 4D.

OffFileReader

Esta clase que crea una malla en 4D a través de la lectura de un archivo **“.off”**¹, con esto conseguimos orientar a datos la construcción de mallas en 4 dimensiones.

De ella heredan las clases *Pentachoron*, *Icositetrachoron*, *Hexadecachoron*, *Hexacosichoron* y *GrandHexacosichoron*, en ellas los polítopos ²se forman a través de la lectura de archivos mencionada anteriormente.

Clases relacionadas con las físicas

Estas clases necesitan heredar de nodos 3D de Godot para aprovechar el motor físico existente.

- **RigidBody4D**: Clase hija de *RigidBody3D* que aplica físicas en 4D a sus nodos hijos. Solo aplica las transformaciones en 4D a los nodos hijos son *Node4D*.
- **CollisionShape4D**: Clase hija de *CollisionShape3D* que mantiene actualizada la malla de colisión.

Explicación breve de cada clase (GDScript)

Clases relacionadas con la escena de físicas

- **drag_and_drop**: Clase encargada de agarrar, arrastrar y soltar los objetos en 4D valiéndose de su *Rigidbody*, como bien su nombre indica.

Clases relacionadas con la escena del visualizador de figuras

- **RotatingPolychoron**: Clase encargada de rotar sobre sí mismo al polícoro del visualizador por medio de clicar y arrastrar el ratón (rotará en los planos **XY**, **XZ**, **YZ**) con la finalidad de poder visualizarlo y manejarlo por completo en el espacio tridimensional.

¹ Los archivos .off fueron creados por el Geometry Center de la Universidad de Minnesota, con el objetivo de describir geometrías con una lista de vértices y caras.

² Los polítopos de cuatro dimensiones son llamados polícoros, son análogos a los polígonos, que son los polítopos de tres dimensiones.

- **UI4DControlPanelVisualizer**: Clase encargada de mostrar una interfaz que es un panel con diferentes funcionalidades para poder interactuar con el polícoro mostrado. Dichas funcionalidades son:
 - Rotar en los planos **XW**, **YW**, **ZW** gracias a un control deslizante.
 - Mostrar el **wireframe** o no de la malla por una casilla de verificación.
 - **Rotar automáticamente** la figura en los planos XW, YW, ZW por una casilla de verificación.
 - **Reiniciar** a la **posición** y **rotación** inicial de la figura por medio de un botón.
- **UIMeshSelector**: Clase encargada de posicionar distintos botones en la interfaz que se encargan de establecer la malla que se seleccione según se desee. Los botones disponibles son los de las mallas de **Pentacoro**, **Teseracto**, **Icositetracoro**, **Hexadecacoro**, **Hexacosicoro**, **Gran hexacosicoro**.
- **ZoomingPolychoron**: Clase encargada de hacer **zoom** a la figura que se está viendo en pantalla cambiando el **FOV**.

Otras clases

- **UIMainMenu**: Clase encargada de hacer funcionar los botones de **jugar** y **salir**.

Bibliografía

Construcción de mallas:

Four-Space Visualization of 4D Objects - index. (s/f). Github.io. Recuperado el 1 de junio de 2026, de https://hollasch.github.io/ray4/Four-Space_Visualization_of_4D_Objects.html

Tsolis, A. (2016, abril 25). 3D visualization of Sutherland-Hodgman algorithm (polygon clipping) - The Portal of Knowledge. Github.io. https://alamot.github.io/sutherland_hodgman_algorithm/

Wikipedia contributors. (2026b, mayo 3). Linear interpolation. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Linear_interpolation&oldid=1352338849

The six regular convex polychora. (n.d.). 4D Polytope Explorer. Retrieved June 1, 2026, from <https://4d.pardesco.com/shapes/regulars/>

Wiki, P. (2026, April 15). *Polytope wiki*. Polytope Wiki. <http://polytope.miraheze.org/wiki/>

Wikipedia contributors. (n.d.). *Polícoro regular convexo*. Wikipedia, The Free Encyclopedia. https://es.wikipedia.org/w/index.php?title=Pol%C3%ADcoro_regular_convexo&oldid=168726385

Construcción de hipercubo:

Wikipedia contributors. (s/f-a). Hipercubo. Wikipedia, The Free Encyclopedia. Recuperado el 22 de mayo de 2026, de <https://es.wikipedia.org/w/index.php?title=Hipercubo&oldid=172101320>

Wikipedia contributors. (s/f-b). Tesseract. Wikipedia, The Free Encyclopedia. <https://es.wikipedia.org/w/index.php?title=Tesseract&oldid=171736493>

Rotaciones:

Wikipedia contributors. (2026a, marzo 27). Rotations in 4-dimensional Euclidean space. Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/w/index.php?title=Rotations in 4-dimensional Euclidean space&oldid=1345748690>

Weisstein, E. W. (s/f). Euler angles. Recuperado el 1 de junio de 2026, de <https://mathworld.wolfram.com/EulerAngles.html>

Visualizador de mallas 4D (Escena proyecto de Godot):

How to have a camera zoom in and out in 3D space. (2025, April 15). Godot Forum. <https://forum.godotengine.org/t/how-to-have-a-camera-zoom-in-and-out-in-3d-space/108313/2>

TheBuffED [@TheBuffED]. (n.d.). *Click and drag 3D rotation in Godot* [Video]. Youtube. Retrieved June 1, 2026, from <https://www.youtube.com/watch?v=fMC9JWg4BMk>

Diagrama de clases

Mermaid. (n.d.). Mermaid.Ai. Retrieved June 1, 2026, from <https://mermaid.ai/open-source/syntax/classDiagram.html>

Mermaid viewer - online diagram editor with live preview. (n.d.). Mermaidviewer.com. Retrieved June 1, 2026, from <https://mermaidviewer.com/>

Online FlowChart & diagrams editor - mermaid live editor. (n.d.). Mermaid.Live. Retrieved June 1, 2026, from <https://mermaid.live/edit>